

Servo & Tracking I: Integral Control for Zero-Error Tracking

Adding an integrator state so the output tracks a reference and rejects constant disturbances – exactly.

Pole placement (`State-Space/PolePlacement.m`) drives the state to zero (regulation) and uses a static reference gain N_r to scale a step. But a static gain leaves a steady-state error the moment a disturbance appears or the plant drifts. **Integral control** augments the plant with the integral of the tracking error, guaranteeing zero steady-state error to a step and rejection of any constant input disturbance. In this tutorial you will:

- augment the plant with an integrator state,
- design the feedback on the augmented system with `place` (or `lqr`), and
- show tracking with zero error and full disturbance rejection (before/after).

Ogata, *Modern Control Engineering*, Ch. 10. Run with `publish('IntegralControl.m')`.

Contents

- Plant
- Augment with an integrator of the tracking error
- Design the state + integral feedback
- After: the servo tracks a step with zero error
- Before vs. after: rejecting a constant input disturbance
- Try it yourself

Plant

$G(s) = \frac{1}{(s+1)(s+2)}$ in state-space; only the output is measured.

```
A = [0 1; -2 -3];  
B = [0; 1];  
C = [1 0];
```

Augment with an integrator of the tracking error

Introduce a state x_i with $\dot{x}_i = r - y = r - Cx$. The augmented system is

$$\begin{bmatrix} \dot{x} \\ \dot{x}_i \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x \\ x_i \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r.$$

```
Aa = [A, zeros(2,1); -C, 0];  
Ba = [B; 0];  
n = size(A,1);
```

Design the state + integral feedback

Place all augmented poles in the left-half plane. The last gain component multiplies the integrator state; `lqr(Aa,Ba,Q,R)` works just as well.

```
Ka = place(Aa, Ba, [-2, -3, -4]);  
K = Ka(1:n); % state feedback  
Ki = Ka(end); % integral gain  
fprintf('State gain K = %s, integral gain Ki = %.4f\n', mat2str(K,4), Ki)
```

```
State gain K = [24 6], integral gain Ki = -24.0000
```

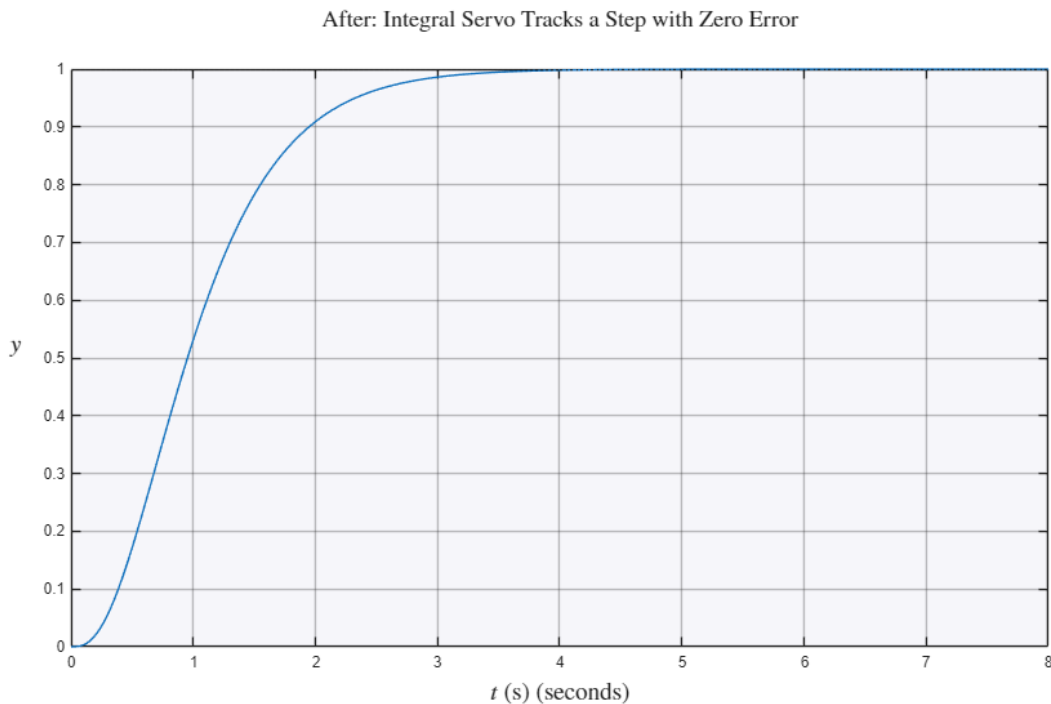
After: the servo tracks a step with zero error

Control law $u = -Kx - K_i x_i$. Build the closed loop from the reference r .

```
Ac1 = [A - B*K, -B*Ki; -C, 0];
Cc1 = [C, 0];
sys_r = ss(Ac1, [0;0;1], Cc1, 0);    % reference -> output

t = 0:0.01:8;
figure
step(sys_r, t)
hold on; yline(1,'k:','HandleVisibility','off'); hold off
grid on
title('After: Integral Servo Tracks a Step with Zero Error','Interpreter','latex','FontSize',15)
ylabel('$y$', 'Interpreter','latex','FontSize',16); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$ (s)', 'Interpreter','latex','FontSize',16)
fprintf('Steady-state tracking error to a step: %.2e\n', 1 - dcgain(sys_r))
```

Steady-state tracking error to a step: 0.00e+00



Before vs. after: rejecting a constant input disturbance

A constant disturbance d enters at the plant input, $\dot{x} = Ax + B(u + d)$. The static- N_r design (no integrator) leaves a permanent offset; the integral servo drives the error back to zero.

```
sys_dist_int = ss(Ac1, [B;0], Cc1, 0);    % disturbance -> output (integral)

Kp = place(A, B, [-3, -4]);
sys_dist_static = ss(A - B*Kp, B, C, 0);    % disturbance -> output (static Nr)

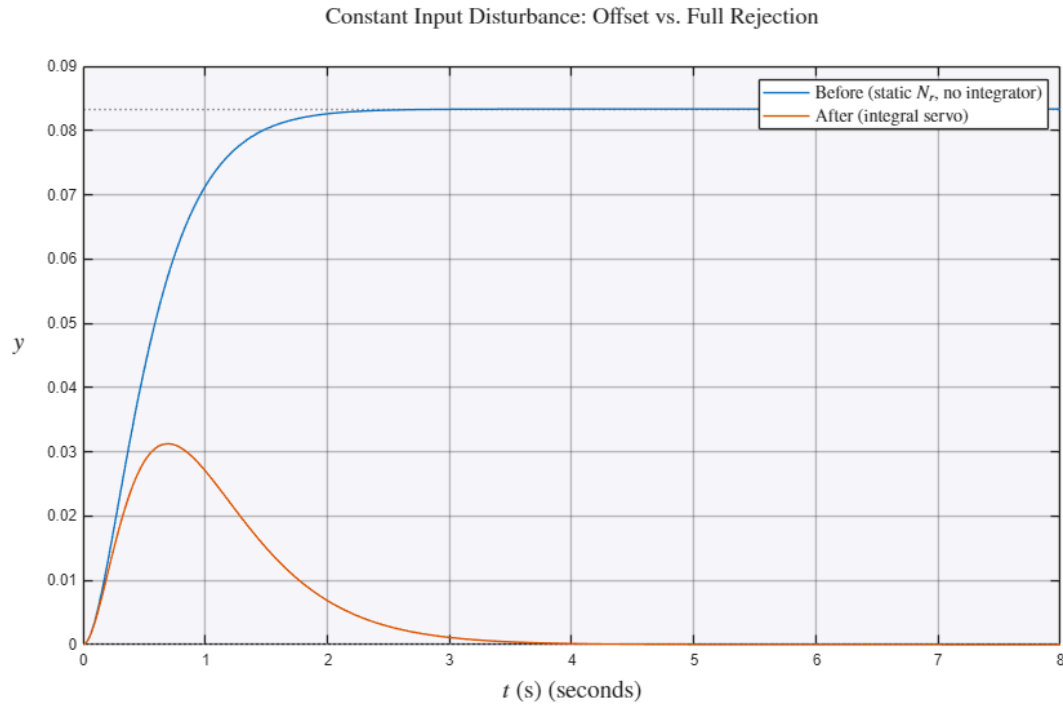
figure
step(sys_dist_static, t)
hold on
step(sys_dist_int, t)
hold off
grid on
legend('Before (static $N_r$, no integrator)', 'After (integral servo)', 'Interpreter','latex','FontSize',12)
title('Constant Input Disturbance: Offset vs. Full Rejection','Interpreter','latex','FontSize',15)
```

```

ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 16); set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$ (s)', 'Interpreter', 'latex', 'FontSize', 16)
fprintf('Disturbance steady-state -- static: %.4f, integral: %.2e\n', ...
    dcgain(sys_dist_static), dcgain(sys_dist_int))

```

Disturbance steady-state -- static: 0.0833, integral: 0.00e+00



Try it yourself

- Change the disturbance size and notice the integral servo always returns to zero error, while the static design's offset scales with it.
- Swap `place` for `lqr(Aa,Ba,diag([1 1 10]),1)` and notice heavier integral weight speed up the disturbance rejection.